

Cartridge & Cloud - Unity Project Setup Guide

Creación y configuración del proyecto desde cero

VRM Games / Blas Luis Rocha González

21/06/2026

1. Objetivo
2. Decisiones iniciales
3. Creación
4. Paquetes
5. Carpetas
6. Assembly Definitions
7. Escenas
8. Project Settings
9. URP
10. Git
11. CI/build local
12. Checklist Sprint 0

Proyecto: Cartridge & Cloud
Título técnico provisional: Cartridge & Cloud
Plataforma inicial: PC / Steam
Motor previsto: Unity 6 LTS / C#
Versión del documento: v0.3
Estado: Preproducción reiniciada desde cero
Fuente conceptual: `Enfoque_v0.6.md`

Regla de interpretación: el proyecto se documenta como una preproducción nueva. No se presupone código, escenas, managers, datos, builds ni sprints implementados. Todo elemento técnico descrito es una especificación o recomendación hasta que exista evidencia de implementación y QA.

1. Objetivo

Crear una base reproducible y limpia para PC/Steam, sin asumir archivos o configuraciones anteriores.

2. Decisiones iniciales

- Unity 6 LTS, versión exacta fijada en `ProjectVersion.txt`.
- Plantilla 3D URP.
- Windows x64.
- Input System.
- TextMeshPro y uGUI.
- AI Navigation.
- Unity Test Framework.
- Localization cuando empiece UI estable.
- Git LFS solo para binarios grandes justificados.

3. Creación

1. Crear proyecto `CartridgeAndCloud` en Unity Hub.
2. Company Name `VRM Games`.
3. Product Name provisional `Cartridge & Cloud`.
4. Configurar Visible Meta Files y Force Text.
5. Añadir `.gitignore` de Unity.
6. Crear repositorio y rama `main` protegida conceptualmente.

7. Hacer build Windows vacío y registrar evidencia.

4. Paquetes

Paquete	Momento
Input System	Sprint 0
AI Navigation	Sprint 0
TextMeshPro	Sprint 0
Test Framework	Sprint 0
Cinemachine	Solo si el spike demuestra valor
Localization	Antes de textos masivos
Addressables	Cuando el volumen de contenido lo justifique
Steamworks	No antes de una build representativa

5. Carpetas

```
Assets/_Project/  
  Art/Characters/Environment/Props/Products  
  Audio/Music/SFX/Mixers  
  Content/Definitions/Balance/Localization  
  Code/Domain/Application/Infrastructure/Presentation/Editor/Tests  
  Prefabs/Furniture/Characters/UI/Systems  
  Scenes/Bootstrap/MainMenu/Store/TestLab  
  Settings  
  UI/Fonts/Icons/Sprites
```

6. Assembly Definitions

VRMGames.CartridgeAndCloud.Domain, .Application, .Infrastructure, .Presentation, .Editor, .Tests.EditM

7. Escenas

Bootstrap carga servicios; MainMenu no mantiene simulación; Store contiene mundo y composición; TestLab permite pruebas aisladas. Evitar managers duplicados por escena.

8. Project Settings

- Color space Linear.
- Input Handling: Input System Package.
- Scripting backend Mono para desarrollo; IL2CPP se valida antes de release.
- API Compatibility según LTS.
- Run in Background configurable.
- Resolution 1920x1080 por defecto.
- VSync y frame cap configurables.

9. URP

Calidad Low/Medium/High, sombras contenidas, postprocesado mínimo, MSAA según perfil y luces baked/mixtas. No introducir efectos caros antes de medir.

10. Git

No versionar Library, Temp, Obj, Logs, UserSettings ni Builds. Versionar Packages, ProjectSettings, Assets y documentación. Commits atómicos con Conventional Commits adaptado.

11. CI/build local

Al menos un script de build reproducible antes del vertical slice. El pipeline debe validar compilación, tests y datos antes de generar artefacto.

12. Checklist Sprint 0

- ☐ Proyecto abre sin warnings críticos.
- ☐ Paquetes resueltos.
- ☐ Build Windows ejecuta.
- ☐ Tests de ejemplo pasan.
- ☐ Escenas creadas.
- ☐

Git limpio.

☐

README y versión actualizados.

☐

Estructura legal y de documentación creada.